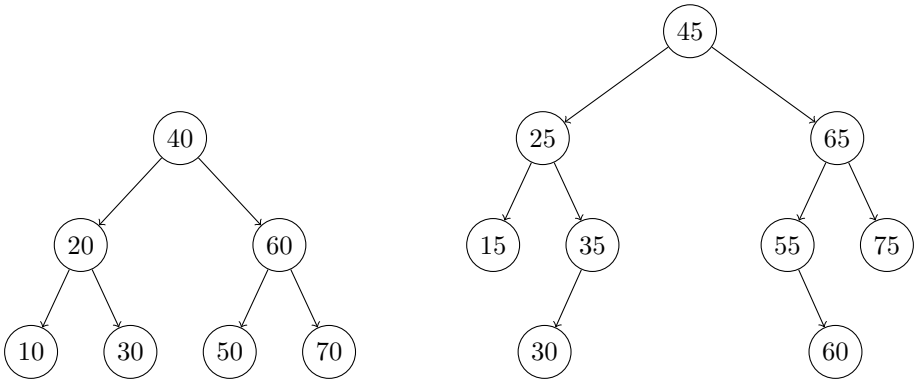
EECE 7205: Fundamentals of Computer Engineering

In-Class Work: Deleting from a Binary Search Tree

Karishma Premnath

Case 1: Deleting the Root



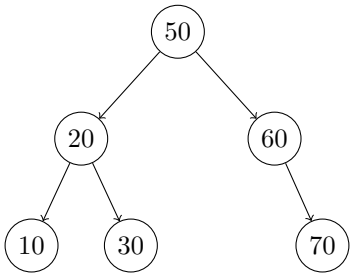
Tree T is on the left with inorder 10 20 30 40 50 60 70, and tree P is on the right with inorder 15 25 30 35 45 55 60 65 75.

1)

- Start at the root 40 and go right once: **visit 60**
- 60 has a left child, so go left: **visit 50**
- 50 has no left child, so the search stops
- **Nodes visited:** 60, 50
- **Successor:** 50
- **Parent of the successor:** 60

2)

40 has two children, so B5 applies and 50 is copied into the root. The original node 50 is a leaf, so B3 removes it by setting `60->left = NULL`. B6 applies as well because the node being deleted is the root.

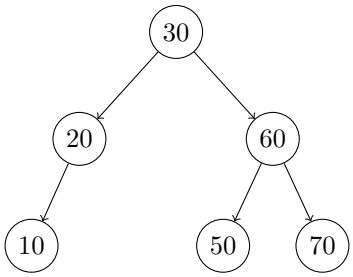


- **Inorder:** 10 20 30 50 60 70
- **Preorder:** 50 20 10 30 60 70

The inorder output is $10 < 20 < 30 < 50 < 60 < 70$, so it is still increasing and B7 holds. Since inorder prints the left subtree, then the node, then the right subtree, getting an increasing list means every node still has smaller values on its left and larger values on its right. The list is also the old one with only 40 taken out, so nothing else was disturbed.

3)

The predecessor is found by going left once to 20, then right as far as possible, which reaches 30. 30 is copied into the root and the old node 30 is removed with `20->right = NULL`.



The new root is 30.

The inorder output does not change. It is 10 20 30 50 60 70 either way. Only the value 40 was deleted, so the tree still holds the same six values, and inorder on a binary search tree always prints its values in increasing order. Using the predecessor instead of the successor only changes which value moves into the root, so the shape changes but the contents do not. The preorder does change, from 50 20 10 30 60 70 to 30 20 10 60 50 70. The predicted output was `After:  10 20 30 50 60 70 (root = 30)` and the run matched it.

```
Node* parent = root;
Node* pred = root->left;
while (pred->right != nullptr)
{
    parent = pred;
    pred = pred->right;              // pred = 30, parent = 20
}
// 2. copy the predecessor's value into the root
root->data = pred->data;             // root now holds 30
// 3. remove the predecessor node
if (parent == root)
    parent->left = pred->left;
else
    parent->right = pred->left;
delete pred;
```

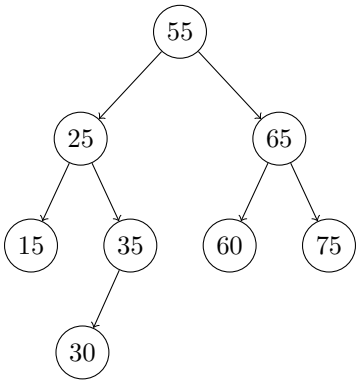
Figure 1: The modified block in `case1_q3.cpp`.

```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 50 60 70   (root = 30)
Preorder after: 30 20 10 60 50 70
```

Figure 2: Output of the run. The root becomes 30 and the inorder traversal is unchanged.

4)

- **Successor:** 55, found by going right once from 45 to 65, then left to 55, which has no left child. Its parent is 65.
- **Rule that removes it:** B4, the one-child rule.
- **Why not B3:** B3 is only for a leaf and sets the parent's link to NULL. 55 is not a leaf, because it has a right child 60. Setting `65->left = NULL` would delete 60 as well and the tree would lose a value. B4 lets the only child move up instead, so 60 becomes the left child of 65.



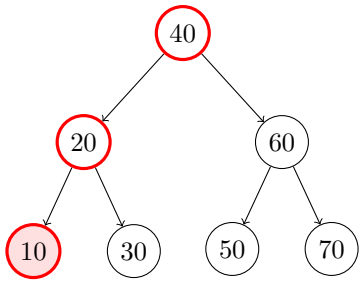
- **Inorder:** 15 25 30 35 55 60 65 75
- **Preorder:** 55 25 15 35 30 65 60 75

The inorder is increasing and is P's original inorder with only 45 removed, so B7 holds and the tree is still a valid binary search tree.

Case 2: Deleting a Leaf

1)

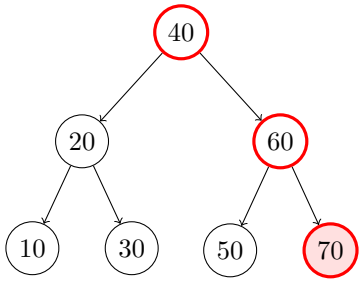
The search starts at the root and keeps going until it finds the key, remembering the parent at every step. For key = 10 the marked path is the route it takes.



- At 40: $10 < 40$, go left
- At 20: $10 < 20$, go left
- At 10: $10 = 10$, found, the loop exits
- **Parent:** 20
- **10 is the left child of 20**, so the code runs `parent->left = nullptr`
- The search makes 3 comparisons, which is $h + 1$ for tree T

2)

With `key = 70` the search goes the other way down the tree.



- At 40: $70 > 40$, go right
- At 60: $70 > 60$, go right
- At 70: $70 = 70$, found, the loop exits
- **Pointer set to NULL:** `60->right`, because 70 is the right child of 60, so the `else` branch of the unlink runs instead of the `if` branch
- **Printed after the deletion:** `After:  10 20 30 40 50 60 (root = 40)`

Only one line changes from `case2.cpp`: `int key = 10;` becomes `int key = 70;`. The unlink code is untouched and still handles both sides. The predicted output was `After:  10 20 30 40 50 60 (root = 40)` and the run matched it.

```
Node* parent = nullptr;
Node* cur = root;
int key = 70;
while (cur != nullptr && cur->data != key)
{
    parent = cur;
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}

if (parent->left == cur) parent->left = nullptr;
else                     parent->right = nullptr;   // 60->right = NULL
delete cur;
```

Figure 3: The modified block in `case2_q2.cpp`.

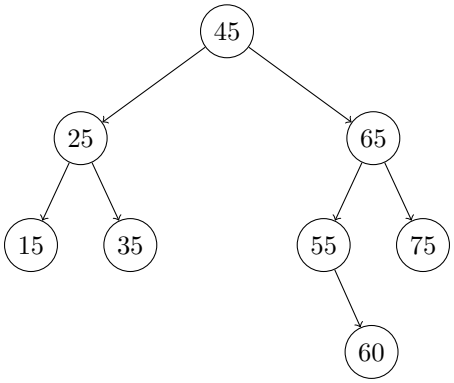
```
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 40 50 60   (root = 40)
```

Figure 4: Output of the run. The leaf 70 is removed and the root is unchanged.

3)

Leaves of tree P: 15, 30, 60 and 75.

30 is a leaf and its parent is 35, so B3 removes it by setting `35->left = NULL`.



Inorder: 15 25 35 45 55 60 65 75

The node that has just become a leaf is 35, since 30 was its only child.

4)

- **Worst case:** $h + 1$ comparisons, one for each node on the longest path from the root down to a leaf, where h is the height measured in edges
- **Tree T:** $h = 2$, so at most 3 comparisons, which is what the search for 10 took
- **Inserting 10, 20, 30, 40, 50, 60, 70 in that order:** every key is larger than the one before it, so each new node is attached on the right and the tree becomes a straight chain with no left children. Here $h = 6$ and the worst case is 7 comparisons

Deleting a leaf is $O(h)$. The unlink itself is only a couple of pointer operations, so the whole cost is the search. In a balanced tree h is about $\log_2 n$ and the deletion is fast, but in the sorted-insert tree the structure degenerates into a linked list with $h = n - 1$ and the deletion becomes $O(n)$. The shape of the tree, not the deletion rule, decides the speed.

Case 3: Deleting a Node with One Child

1)

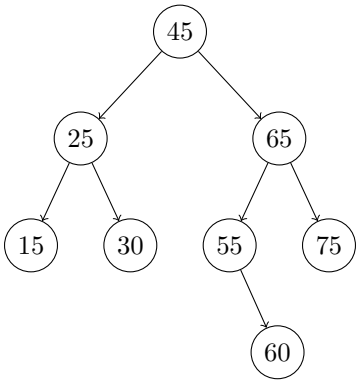
- **35** has exactly one child, 30, and it is a **left** child
- **55** has exactly one child, 60, and it is a **right** child

Every other node of P has either two children (45, 25, 65) or none (15, 30, 60, 75). Tree T has no node with exactly one child to start with, which is why `case3.cpp` first removes 50 so that 60 is left with a single child. Tree P already contains such nodes, so no setup step is needed.

2)

Deleting 35 from P:

- **Parent:** 25
- **Child that moves up:** 30
- **Pointer that changes:** `25->right = 30`, because 35 was the right child of 25



Inorder: 15 25 30 45 55 60 65 75

The predicted output was `After:  15 25 30 45 55 60 65 75 (root = 45)` and the run matched.

```
int keys[] = {45, 25, 65, 15, 35, 55, 75, 30, 60};
Node* root = nullptr;
for (int k : keys) root = insert(root, k);

cout << "Before: ";
printInorder(root);
cout << "  (root = " << root->data << ")" << endl;

// 1. search for 35, remembering its parent
Node* parent = nullptr;
Node* cur = root;
int key = 35;
while (cur != nullptr && cur->data != key)
{
    parent = cur;
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
// 2. its only child takes its place
Node* child = (cur->left != nullptr) ? cur->left : cur->right;
cout << "parent = " << parent->data
     << ", child that moves up = " << child->data << endl;
if (parent->left == cur) parent->left = child;
else                     parent->right = child;
delete cur;
```

Figure 5: The modified block in `case3_q2.cpp`.

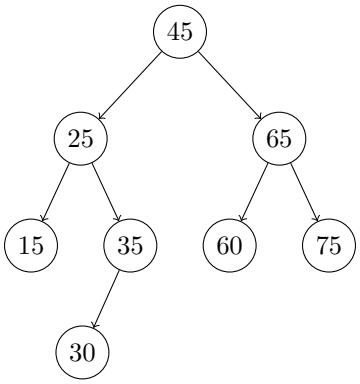
```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
parent = 25, child that moves up = 30
After:  15 25 30 45 55 60 65 75   (root = 45)
```

Figure 6: Output: 30 moves up into the place of 35.

3)

Starting again from the original P and deleting 55:

- **Parent:** 65
- **Child that moves up:** 60
- **Pointer that changes:** `65->left = 60`, because 55 was the left child of 65



Inorder: 15 25 30 35 45 60 65 75

Why `parent->left` this time. The branch depends on which side the deleted node hangs off its parent, not on which side its own child is on. 35 was the right child of 25, so the `else` branch ran and wrote `parent->right`. 55 is the left child of 65, so the test `if (parent->left == cur)` succeeds and the `if` branch runs instead. The child's own side is irrelevant: 35's child was on the left and 55's child is on the right, yet in both cases the branch chosen is the opposite of that.

```
int keys[] = {45, 25, 65, 15, 35, 55, 75, 30, 60};
Node* root = nullptr;
for (int k : keys) root = insert(root, k);

cout << "Before: ";
printInorder(root);
cout << "  (root = " << root->data << ")" << endl;


Node* parent = nullptr;
Node* cur = root;
int key = 55;
while (cur != nullptr && cur->data != key)
{
    parent = cur;
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
// 2. its only child takes its place
Node* child = (cur->left != nullptr) ? cur->left : cur->right;
cout << "parent = " << parent->data
     << ", child that moves up = " << child->data << endl;
if (parent->left == cur) parent->left = child;
else                     parent->right = child;
delete cur;
```

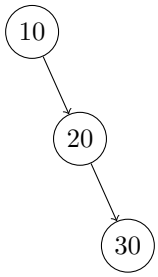
Figure 7: The modified block in `case3_q3.cpp`.

```
Before: 15 25 30 35 45 55 60 65 75   (root = 45)
parent = 65, child that moves up = 60
After:  15 25 30 35 45 60 65 75   (root = 45)
```

Figure 8: Output: 60 moves up into the place of 55.

4)

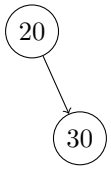
Inserting 10, 20, 30 in that order gives a chain, because each key is larger than the one before it and is attached on the right.



- **Rule: B6**. The root follows the same rules, but when it has 0 or 1 child the root pointer itself has to change, since there is no parent whose link could be updated instead.
- **New root:** 20

What goes wrong in `case3.cpp` unchanged. `parent` is initialised to `nullptr`, and when the key being deleted is the root the while loop never runs its body, so `parent` is still `nullptr` afterwards. The code then reaches `if (parent->left == cur)` and dereferences a null pointer, which crashes the program. Even with that line guarded, the code only ever writes into a parent's link, so nothing would update `root` and it would be left pointing at the node that was just deleted.

The fix is one extra branch placed before the other two, so that the root pointer is reassigned when there is no parent:



Inorder: 20 30

```cpp
int keys[] = {10, 20, 30};
Node* root = nullptr;
for (int k : keys) root = insert(root, k);

cout << "Before: ";
printInorder(root);
cout << "  (root = " << root->data << ")" << endl;


Node* parent = nullptr;
Node* cur = root;
int key = 10;
while (cur != nullptr && cur->data != key)
{
    parent = cur;
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}

Node* child = (cur->left != nullptr) ? cur->left : cur->right;


if (parent == nullptr)
    root = child;
else if (parent->left == cur)
    parent->left = child;
else
    parent->right = child;
delete cur;
```

Figure 9: The modified block in `case3_q4.cpp`. The `parent == nullptr` branch is what handles B6.

```
Before: 10 20 30   (root = 10)
After:  20 30   (root = 20)
```

Figure 10: Output: the root changes from 10 to 20.

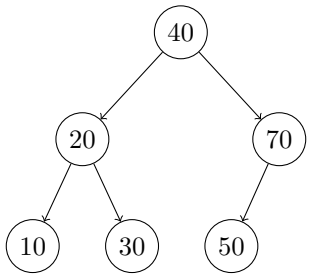
Case 4: Deleting a Node with Two Children

1)

With `key = 60` the search finds 60 straight away, since $60 > 40$ sends it right. Then `parent = cur` and `succ = cur->right` gives 70. Because `70->left` is NULL the while loop never runs, so `parent` is never reassigned.

- `succ` = **70**
- `parent` = **60**, which is `cur` itself
- **Branch taken:** the `if (parent == cur)` branch, so `parent->right = succ->right` sets `60->right = NULL`
- **Printed:** `After:  10 20 30 40 50 70 (root = 40)`

That branch runs because the successor turned out to be `cur`'s own right child, so the loop body never executed and `parent` still equals `cur`. The `else` branch is for when the walk went left at least once, in which case the successor hangs off some parent's left link instead.



Inorder: 10 20 30 40 50 70

The only change from `case4.cpp` is `int key = 20;` becoming `int key = 60;`. The predicted output was `After:  10 20 30 40 50 70 (root = 40)` and the run matched it.

```cpp
Node* cur = root;
int key = 60;
while (cur != nullptr && cur->data != key)
{
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}

Node* parent = cur;
Node* succ = cur->right;                // 70
while (succ->left != nullptr)
{
    parent = succ;
    succ = succ->left;
}
cout << "succ = " << succ->data << ", parent = " << parent->data
     << "  (parent == cur is " << (parent == cur ? "true" : "false") << ")" << endl;

cur->data = succ->data;

if (parent == cur)
    parent->right = succ->right;        // 60->right = NULL
else
    parent->left = succ->right;
delete succ;
```

Figure 11: The modified block in `case4_q1.cpp`.

```
Before: 10 20 30 40 50 60 70   (root = 40)
succ = 70, parent = 60  (parent == cur is true)
After:  10 20 30 40 50 70   (root = 40)
```

Figure 12: Output: 70 is copied into the node that held 60 and the old node 70 is removed.

2)

The successor is found by going right once and then left as far as possible, and that loop only stops when `succ->left` is NULL. So by construction the successor cannot have a left child.

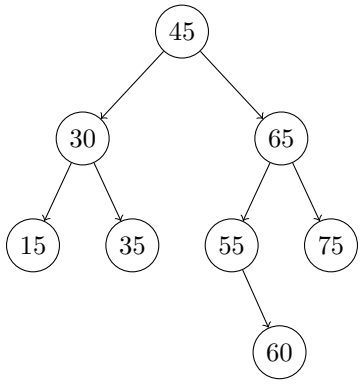
The reason behind the construction is that the successor is the smallest value in the right subtree. If it had a left child, that child would be smaller than it and still larger than the node being deleted, so the child would be the real successor instead. A node with a left child can never be the smallest one in its subtree.

What would go wrong in step 4. That step reattaches only `succ->right`, writing either `parent->right = succ->right` or `parent->left = succ->right`. If the successor had a left child, that whole left subtree would never be reattached to anything. It would be cut out of the tree, its values lost from the traversal, and the memory leaked, because `delete succ` frees only the successor node itself and not the subtree hanging below it.

3)

Deleting 25 from P. It has two children, 15 and 35, so B5 applies.

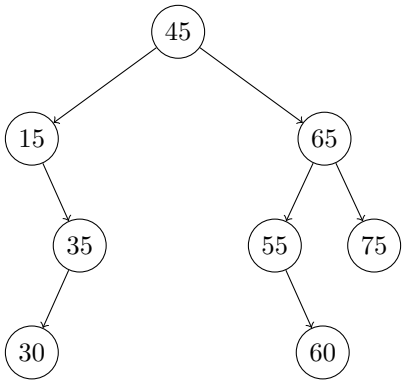
- **Successor:** 30, found by going right once to 35, then left to 30, which has no left child
- **Its parent:** 35
- **Pointer changed:** `35->left = NULL`, through the `else` branch, since `parent` (35) is not `cur`



Inorder: 15 30 35 45 55 60 65 75

4)

Deleting 25 from the original P with the predecessor instead. Going left once reaches 15, which has no right child, so the walk stops there. The predecessor is 15 and its parent is the node holding 25, so `parent == cur` and the first branch runs, clearing that node's left link.



Inorder: 15 30 35 45 55 60 65 75

- **Is it the same tree as question 3?** No. The contents are identical but the shapes differ. The preorder traversals make this visible: question 3 gives 45 30 15 35 65 55 60 75 while question 4 gives 45 15 35 30 65 55 60 75.
- **Are both valid binary search trees?** Yes. Both inorder traversals are increasing, and both equal P's original inorder with exactly 25 removed, so B7 holds in both cases.

Case 5: The Key Is Not in the Tree

1)

Searching tree T for 25:

- At 40: $25 < 40$, go left
- At 20: $25 > 20$, go right
- At 30: $25 < 30$, go left, and that link is NULL
- **Comparisons with node values:** 3
- **Stops at:** the **left** link of node **30**

That NULL position is exactly where 25 would go if it were inserted, which is why searching and inserting share the same loop.

2)

- **Key 5:** $5 < 40$ left, $5 < 20$ left, $5 < 10$ left. 3 comparisons, stops at the left link of 10.
- **Key 45:** $45 > 40$ right, $45 < 60$ left, $45 < 50$ left. 3 comparisons, stops at the left link of 50.
- **Key 75:** $75 > 40$ right, $75 > 60$ right, $75 > 70$ right. 3 comparisons, stops at the right link of 70.

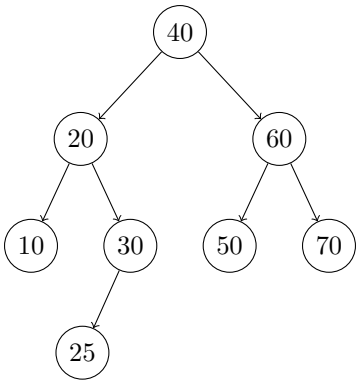
None of them needs more than the others. All three take exactly 3 comparisons, the same as the search for 25.

This follows directly from the height. Tree T is perfectly balanced with $h = 2$, so every path from the root down to a NULL link passes through exactly $h + 1 = 3$ nodes. An unsuccessful search always runs all the way to the bottom, so in a perfectly balanced tree every failed search costs the same. In general the worst case is $h + 1$ comparisons, and in T every search reaches that worst case. In a lopsided tree the counts would vary, anywhere from 1 up to $h + 1$.

3)

Once `cur` becomes NULL the search has lost track of where it came from, so the loop has to remember the parent. The new node is then attached to the correct side of that parent, choosing left or right by the same comparison the search used.

25 is inserted as the **left child of 30**, which is the NULL link where the search stopped.



Inorder: 10 20 25 30 40 50 60 70

The changes from `case5.cpp` are adding `Node* parent = nullptr;`, adding `parent = cur;` inside

the loop, and replacing the "nothing to delete" message with the insert block. The predicted output was `After:  10 20 25 30 40 50 60 70 (root = 40)` and the run matched it.

```
Node* parent = nullptr;
Node* cur = root;
int key = 25;
while (cur != nullptr && cur->data != key)
{
    parent = cur;
    if (key < cur->data) cur = cur->left;
    else                 cur = cur->right;
}
if (cur == nullptr)
{

    Node* fresh = new Node(key);
    if (parent == nullptr)        root = fresh;        // the tree was empty
    else if (key < parent->data)  parent->left = fresh;
    else                          parent->right = fresh;
    cout << key << " was not found, inserted as the "
         << (key < parent->data ? "left" : "right")
         << " child of " << parent->data << endl;
}
```

Figure 13: The modified block in `case5_q3.cpp`.

```
Before: 10 20 30 40 50 60 70   (root = 40)
25 was not found, inserted as the left child of 30
After:  10 20 25 30 40 50 60 70   (root = 40)
```

Figure 14: Output: 25 is inserted at the NULL link where the search stopped.

4)

If the tree is empty then `root` is `nullptr`, so `root->data` dereferences a null pointer and the program crashes. It crashes on the very first line that reads `root`, the `Before:` line, so the search never runs and the "nothing to delete" message is never reached. The search loop itself would have been safe, since the test `cur != nullptr` fails immediately and `cur` is never dereferenced.

The check has to come before anything reads `root`:

```
if (root == nullptr)
{
    cout << "the tree is empty" << endl;
    return 0;
}
```

Figure 15: The check added at the start of `case5_q4.cpp`.

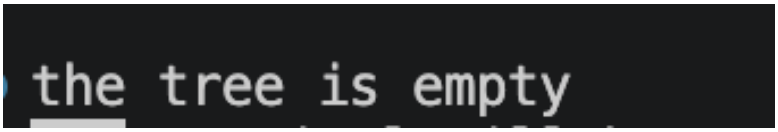


Figure 16: Output when the tree is empty.

Rule: B2. The search reaches NULL, so the key is not in the tree and nothing changes. An empty tree is the extreme case of it, where the search reaches NULL immediately and makes zero comparisons.

Bonus: One Function for Every Case

The function `Node* deleteNode(Node* root, int key)` returns the possibly new root and handles all five cases in one place. It was tested on tree T by deleting 40, 10, 60 and 25 one after another, printing the inorder traversal after each deletion.

```
Node* deleteNode(Node* root, int key)
{
    if (root == nullptr) return nullptr;

    if (key < root->data)
    {
        root->left = deleteNode(root->left, key);
        return root;
    }
    if (key > root->data)
    {
        root->right = deleteNode(root->right, key);
        return root;
    }

    if (root->left == nullptr && root->right == nullptr)
    {
        delete root;
        return nullptr;
    }
    if (root->left == nullptr)
    {
        Node* child = root->right;
        delete root;
        return child;
    }
    if (root->right == nullptr)
    {
        Node* child = root->left;
        delete root;
        return child;
    }

    Node* succ = root->right;
    while (succ->left != nullptr) succ = succ->left;
    root->data = succ->data;
    root->right = deleteNode(root->right, succ->data);
    return root;
```

Figure 17: The `deleteNode` function.

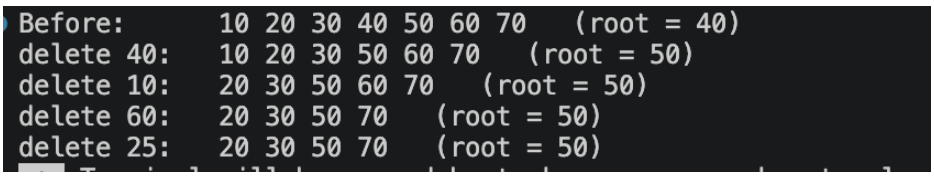


Figure 18: Output. Each of the four deletions exercises a different case.

How the cases are covered

- **B1** needs no `parent` variable. The recursion keeps the link, because `root->left = deleteNode(root->left, key)` writes the returned subtree straight back into the parent's pointer.
- **B2** is the first line. If the search runs off the bottom of the tree the function returns `nullptr` and the tree is rebuilt exactly as it was.
- **B3** is the branch where both children are NULL. The node is freed and `nullptr` goes back to the parent.
- **B4** is the two branches that return the surviving child, so that child is linked into the parent in place of the deleted node.
- **B5** copies in the inorder successor and then calls itself on the right subtree to remove the successor. That call always lands in the B3 or B4 branch, because a successor never has a left child.
- **B6** is handled by the return value. The caller writes `root = deleteNode(root, key)`, so when the root itself is removed the root pointer is reassigned. This is the same problem that Case 3 question 4 hit with `parent == nullptr`, solved here by the structure of the function rather than by a special case.

What each test deletion exercises

- **40** is the root with two children, so B5 and B6
- **10** is a leaf, so B3
- **60** has only a right child by that point, so B4
- **25** was never in the tree, so B2 and the tree is left unchanged

Time complexity

The function is $O(h)$, where h is the height of the tree. It walks one path from the root down to the node being deleted, at most $h+1$ steps. When that node has two children the while loop walks down to the successor, and the recursive call then walks from `root->right` down to that successor a second time in order to remove it, so the worst case is roughly $2h$ steps. That is a fixed number of descents of the tree, so the cost is still $O(h)$.

In terms of the number of nodes n, a balanced tree has $h \approx \log_2 n$ and deletion is $O(\log n)$, while a tree built from sorted inserts degenerates into a chain with $h = n-1$ and deletion becomes $O(n)$. The shape of the tree decides the cost, not the deletion rule.